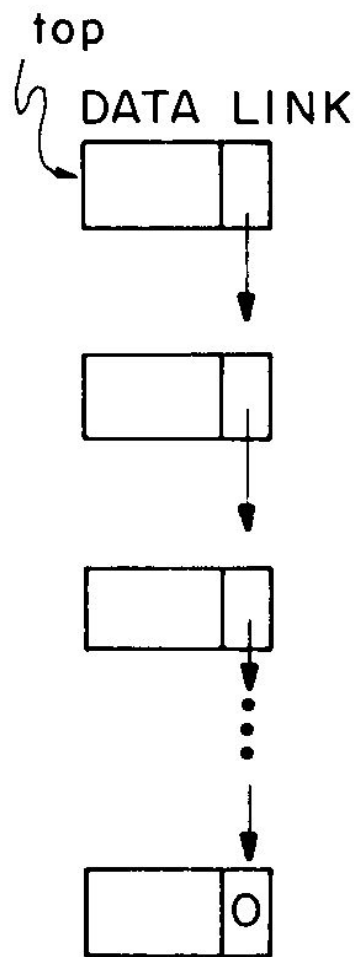


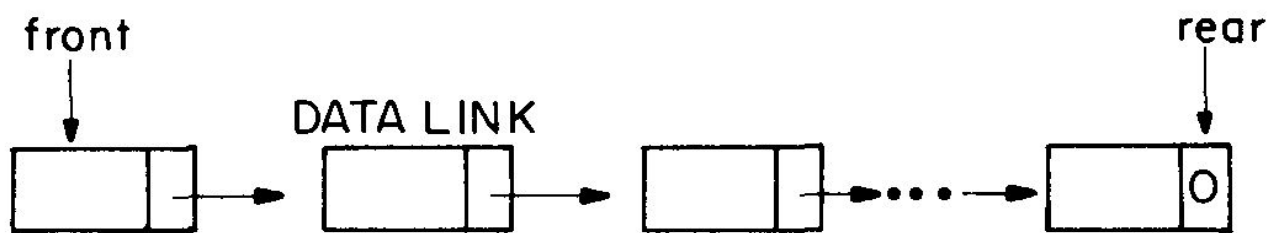
Linked Lists

4.2 LINKED STACKS AND QUEUES

- Sequential representation of stacks and queue proved efficient if we had only one stack or one queue.
- However, when several stacks and queues co-exist, there was no efficient way to represent them sequentially.
- In this section we present a good solution to this problem using **linked lists**.
- Figure 4.5 shows a **linked stack** and a **linked queue**.



(a) Linked Stack



(b) Linked Queue

Figure 4.5

- **Direction of links** for both the stack and queue are such as to **facilitate easy insertion** and deletion of nodes.
- In the case of figure 4.5(a), one can easily **add** a node at the top or **delete** one from the **top**.
- In fig4.5(b), one can easily add a node at the rear and both **addition** and **deletion** can be performed at the **front**
- To represent n stacks and m queues simultaneously, then the following set of algorithms and initial conditions will serve the purpose

$$\begin{array}{ll}
 T(i) & = \text{Top of } i\text{th stack} & 1 \leq i \leq n \\
 F(i) & = \text{Front of } i\text{th queue} & 1 \leq i \leq m \\
 R(i) & = \text{Rear of } i\text{th queue} & 1 \leq i \leq m
 \end{array}$$

Initial conditions:

$$\begin{array}{ll}
 T(i) & = 0 & 1 \leq i \leq n \\
 F(i) & = 0 & 1 \leq i \leq m
 \end{array}$$

Boundary conditions:

$$\begin{array}{ll}
 T(i) & = 0 & \text{iff stack } i \text{ empty} \\
 F(i) & = 0 & \text{iff queue } i \text{ empty}
 \end{array}$$

procedure *ADDS*(*i*, *Y*)

 //add element *Y* onto stack *i*//

call *GETNODE*(*X*)

$DATA(X) \leftarrow Y$ //store data value *Y* into new node//

$LINK(X) \leftarrow T(i)$ //attach new node to top of *i*-th stack//

$T(i) \leftarrow X$ //reset stack pointer//

end *ADDS*

procedure *DELETES*(*i*, *Y*)

 //delete top node from stack *i* and set *Y* to be the *DATA* field of
 this node//

if $T(i) = 0$ **then** **call** *STACK_EMPTY*

$X \leftarrow T(i)$ //set *X* to top node of stack *i*//

$Y \leftarrow DATA(X)$ //Y gets new data//

$T(i) \leftarrow LINK(X)$ //remove node from top of stack *i*//

call *RET*(*X*) //return node to storage pool//

end *DELETES*

procedure *ADDQ*(*i*, *Y*)

add Y to the i th queue

call *GETNODE*(*X*)

$$DATA(X) \leftarrow Y; LINK(X) \leftarrow 0$$

```

if  $F(i) = 0$  then  $[F(i) \leftarrow R(i) \leftarrow X]$  //the queue was empty//
else  $[LINK(R(i)) \leftarrow X; R(i) \leftarrow X]$  //the queue was
                                         not empty//

```

end ADDQ

procedure *DELETEQ*(*i*, *Y*)

⧻delete the first node in the i th queue, set Y to its $DATA$ field⧻

if $F(i) = 0$ then call *QUEUE_EMPTY*

else $[X \leftarrow F(i); F(i) \leftarrow LINK(X)$

```

//set X to front node//

```

$Y \leftarrow DATA(X); \text{call } RET(X)]$ $\parallel$ remove data
and return node $\parallel$

end *DELETEQ*

- The above algorithm to the n -stack, m -queue problem is both **computationally and conceptually simple**.
- There is **no need to shift** stacks or queues around to make space.
- Computation can proceed so long as there are free nodes.
- Though additional space is needed for the link fields, the **cost is no more than a factor of 2**.
- Sometimes the DATA field does not use the whole word and it is **possible to pack** the **LINK and DATA fields** into the **same word**.
- In such a case the **storage** requirements for **sequential and linked representations** would be the **same**.
- The **overhead** incurred by the **storage** for the links in linked lists must be overridden by:
 - (i) the virtue of being able to represent **complex lists** all within the **same array**, and
 - (ii) the **computing time** for manipulating the lists is **less** than for **sequential representation**.

4.3 THE STORAGE POOL

- The **storage pool** contains all nodes that are **not currently being used**.
- **RET** and a **GETNODE** procedure will return and remove nodes to and from the pool.
- The first problem to be solved when using linked allocation is exactly **how a node is to be constructed**
- The **no**, and **size** of the **data fields** will **depend** upon the **kind of problem**.
- The **no of pointers** will depend upon the **structural properties** of the data and the **operations** to be performed.
- The amount of **space** to be allocated **for** each **field** depends partly on the **problem** and partly on the **addressing characteristics** of the machine.

- The variable **AV** must initially be set to **one**
- Assume it is a **global variable**

procedure *GETNODE(I)*

 //I is set as a pointer to the next available node//

if $AV > n$ **then call** *NO__MORE__NODES*

$I \leftarrow AV$

$AV \leftarrow AV + 1$

end *GETNODE*

- The **main idea** is to initially link together all of the available nodes in a single list.
- This list will be singly linked where we choose any one of the possible link fields as the field through which the available nodes are linked.
- This must be done at the beginning of the program, using a procedure such as

procedure *INIT*(*n*)

 //initialize the storage pool, through the *LINK* field, to contain nodes
 with addresses 1,2,3, ..., *n* and set *AV* to point to the first node
 in this list//

for *i* ← 1 **to** *n* − 1 **do**

$LINK(i) \leftarrow i + 1$

end

$LINK(n) \leftarrow 0$

$AV \leftarrow 1$

end *INIT*

This procedure gives us the following list:

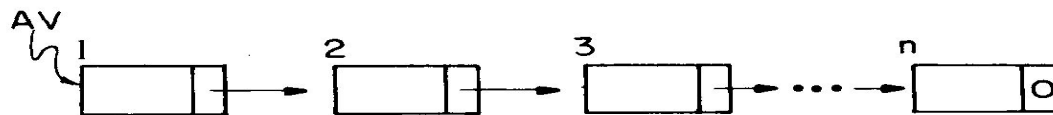


Figure 4.6 Initial Available Space List

- Once **INIT** has been executed, the program can **begin to use nodes**.
- Every time a **new node** is needed, a call to the **GETNODE** procedure is made.
- GETNODE examines the list *AV* and **returns the first node** on the list.
- This is accomplished by the following:

procedure *GETNODE*(*X*)

 // *X* is set to point to a free node if there is one on *AV* //

if *AV* = 0 **then call** *NO__MORE__NODES*

X ← *AV*

AV ← *LINK*(*AV*)

end *GETNODE*

procedure *RET*(*X*)

 // *X* points to a node which is to be returned to the available space
 list //

LINK(*X*) ← *AV*

AV ← *X*

end *RET*

- Because *AV* must be used by several procedures, assume it is a global variable.
- Whenever the programmer knows he can return a node he uses procedure *RET* which will insert the new node at the front of list *AV*.
- This makes **RET efficient** and implies that the **list AV** is used as a **stack** since the last node inserted into *AV* is the first node removed (**LIFO**).